

**SecureLockTS: A Self-Lockdown Application for Real-Time Attack Detection and
Automated System Protection**

Khushi, Taiba, Heer

1. Introduction

Background and Context

Contemporary web apps are more vulnerable to cybersecurity threats, especially injection attacks, authentication bypass, and insecure use of API. In the OWASP Top 10:2025, injection vulnerabilities are categorized as one of the most critical risks associated with the web application (OWASP Foundation, 2025). Due to the increased integration of systems via APIs and cloud security, an attacker abuses lax input validation and incorrect authorization processes. Conventional defensive strategies tend to take advantage of external surveillance instead of the secure coding practices, which are built in. In this way, it is important to develop automated detection and response mechanisms into the application.

Problem Statement

A large number of small and middle-level applications do not have internal security mechanisms that are capable of identifying and responding to attacks in real time. In case of malicious input processing, the system can still run in a trusted state, and attackers can escalate privileges or manipulate data. Such a discontinuity between vulnerability identification and system reaction poses a serious risk.

Purpose and Objectives

This project will be aimed at designing and analyzing a self-lockdown web application, which can identify suspicious activity and take defensive measures, including the automatic disabling of affected modules or the temporary lock of the system.

The objectives are:

1. Create a basic web app with TypeScript.
2. Add controlled vulnerabilities to be analyzed.
3. Use the tools of statistical analysis to find the weaknesses.
4. Install an algorithmic detection module that will initiate system lockdown on occasions of detection of suspicious input patterns.

TypeScript is chosen due to its use of static typing that minimizes runtime errors and enhances code maintainability, making it more straightforward to apply secure coding principles.

Scope

This project will concentrate on injection vulnerabilities and authentication vulnerabilities in a simplified web-based application created by the group. It will compare secure coding practices and the findings of the static analysis. It will not discuss enterprise implementation, high-tech AI-driven detection, or cloud infrastructure security.

Significance and Impact

This project will help developers, students, and organizations by showing how proactive security mechanisms have the potential to minimize harm caused by cyberattacks. Studies indicate that insecure coding practices remain a contributor to breaches (Bathgate, 2026). The system reduces risk exposure as it incorporates automated lockdown solutions and strengthens a safe development lifecycle culture.

2. System or Codebase Description

The system will be a simplified TypeScript web application written in Node.js. It will involve a login, API endpoints, and user input fields. The application will be used to re-create the

real-life scenarios when the attackers are trying to inject commands or bypass authentication. The detection element will observe the input patterns and the unusual access attempts and lockdown processes on the occurrence of thresholds.

3. Target Vulnerabilities

The project will target three main categories of vulnerabilities, which are command/code injection, authentication and authorization vulnerabilities, and input validation weaknesses. Command or code injection is a technique whereby unsanitized user input is run as a system command, and may allow attackers to execute malicious instructions on the server. Weak authentication and authorization weaknesses occur when validation of the sessions and access control is incorrectly performed, allowing unauthorized individuals to have privileged access. Input validation attacks are a failure by applications to sanitize or otherwise manipulate the data provided by the user prior to processing it. These forms of vulnerabilities are always listed as the essential web security threats within the OWASP Top 10 (OWASP Foundation, 2025).

4. Planned Static Analysis Tools

To analyze the security posture of the application, some of the tools to be used during the development process will be the static analysis tools. Unsafe coding patterns will be identified, the coding standards will be enforced, and the possible security concerns will be revealed much earlier in the coding process with the help of ESLint with the TypeScript plug-in. SonarQube will offer advanced static application security testing (SAST), such as taint analysis to trace unknown input flows and track injection or authentication vulnerabilities (SonarSource, 2025). Also, the TypeScript compiler strict mode will be enabled to enhance type-checking and minimize runtime errors and unsafe operations, which might introduce security vulnerabilities to the system.

5. Initial Research

The existence of injection vulnerabilities has had a widespread real-world impact. To illustrate, the Equifax attacker took advantage of an unpatched vulnerability in a web application, and millions of users were impacted. According to industry reports, breaches are still caused by insecure coding (Kobie, 2025). Such attacks show the necessity of active, built-in security measures.

6. Timeline

- Week 1: Planning and architectural design of the project.
- Week 2: Build a fundamental application using TypeScript.
- Week 3: Implement and report test vulnerabilities.
- Week 4: Operate static analysis tools and analyze results.
- Week 5: Use the detection and lockdown algorithm.
- Week 6: Testing, documentation, and final report preparation.

References

Bathgate, R. (2026, February 17). *The vast majority of breaches are enabled by preventable gaps and identity weaknesses, says Palo Alto Networks*. IT Pro.

<https://www.itpro.com/security/cyber-attacks/vast-majority-breaches-enabled-preventable-gaps-identity-weaknesses-palo-alto-networks?utm>

Kobie, N. (2025, August 26). *74% of companies admit insecure code caused a security breach*.

IT Pro. <https://www.itpro.com/security/74-percent-of-companies-admit-insecure-code-caused-a-security-breach?utm>

OWASP Foundation. (2025). *OWASP Top 10:2025 RC1 – A05: Injection*.

https://owasp.org/Top10/2025/A05_2025-Injection/

Sonar. (2025). *Advanced code security tool for developers with SonarQube*.

<https://www.sonarsource.com/solutions/security/>